

Module 5

SPI — Protocol + RTL + Basic Testbench

Learning objectives

- SPI mode 0: SCLK idle low; capture on rising edge; change on falling edge; CS_N active low for fram...
- RTL block architecture: ``clk_div`` + ``spi_master`` hierarchy, FSM, and Mode 0 timing on the wire.
- Baseline verification: Directed stimulus, ``done``-based checks, and optional waveform review (8 SCLK...
- Path to Module 6: Same DUT, UVM agent with bus monitor on SCLK/MOSI/CS_N.

Prerequisites

- Modules 1–4 (spec→RTL, UVM, UART baseline, UART UVM).
- Verilator, Make, C++ compiler. No UVM required for spi_baseline.

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["SPI Protocol (Mode 0)"]

T2["Hands-on testbench (spi_base)"]

T1 --> T2

T2 --> N["SPI UVM+SV"]

Understand the SPI protocol (mode 0, signals), translate it to RTL (SPI master, clk_div), and verify with a **basic** (non-UVM) directed te...

Overview

- Module 5 is the second **protocol** module (after UART):

How to learn this module

Suggested learning path

- Read the detailed SPI learning guide:
SPI_LEARNING_GUIDE.md — what SPI is, what kind of protocol (s...
- Read the protocols + UVM overview:
LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 4.
When to Use Ba...

Design architecture

1. Block hierarchy

- top_spi_baseline → `clk\_div` +
`spi\_master`; C++ supplies
`clk`/`rst\_n`.

Rtl Architecture

Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
CPP["sim_main.cpp\ncbk, rst_n"]
DIV["clk_div\ncbk_div_tick"]
SPI["spi_master\nMode 0 MSB first"]
CPP --> DIV
DIV --> SPI

2. SPI master RTL

- clk_div: `clk\_div\_tick` every
`DIVIDER` clocks.
- spi_master: Mode 0; `cs\_n`
frames; MSB-first on `mosi`;
`done` when complete.
- Outputs: `sclk` (idle low),
`mosi`, `cs\_n` — no `miso` in
baseline.

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

STIM["initial: start, data_in\n0x55, 0xAA"] --> M["spi_master"]

M --> WAIT["wait done"]

WAIT --> PASS["SPI baseline PASS"]

NOTE["Optional: VCD check\n8 SCLK edges, Mode 0"] --> PASS

3. Mode 0 timing on the wire

- Capture on rising SCLK; change MOSI on falling SCLK; 8 edges per byte.

Testing Methods

Diagram: Testing Methods
(render with mmdc when available)
flowchart LR
STIM["start + data_in"] --> SPI["spi_master"]
SPI --> DONE["wait done"]
DONE --> PASS["baseline PASS"]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

CPP["sim_main.cpp\nclk, rst_n"]

DIV["clk_div\nclk_div_tick"]

SPI["spi_master\nMode 0 MSB first"]

CPP --> DIV

DIV --> SPI

Module 5: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

STIM["initial: start, data_in\n0x55, 0xAA"] --> M["spi_master"]

M --> WAIT["wait done"]

WAIT --> PASS["SPI baseline PASS"]

NOTE["Optional: VCD check\n8 SCLK edges, Mode 0"] -.-> PASS

Module 5: stimulus, observation, and checking.

spi_master — Mode 0 shift

```
// Simple SPI master (mode 0, single slave, 8-bit transfers).
// This is a teaching example: it assumes a clean clk_div_tick input
// that
// controls SCLK edges and bit timing.
//
// Mode 0 (CPOL=0, CPHA=0):
//   - SCLK idles low.
//   - Data is captured on the rising edge of SCLK.
//   - Data is changed on the falling edge of SCLK.

`timescale 1ns/1ps

module spi_master (
    input wire      clk,
    input wire      rst_n,          // Active-low async reset
    input wire      clk_div_tick, // Divider pulse controlling SCLK
    output reg [7:0] data_out,
    output reg [7:0] data_in,
    output reg       toggling
    // Data is shifted out on the falling edge of SCLK.
    // Data is shifted in on the rising edge of SCLK.
```

clk_div — SPI clock

```
// Simple clock divider for SPI SCLK timing.  
// Generates a 1-cycle-wide clk_div_tick pulse every DIVIDER cycles of  
clk.
```

```
`timescale 1ns/1ps
```

```
module clk_div #(  
    parameter int DIVIDER = 8  
) (  
    input  wire clk,  
    input  wire rst_n,          // Active-low async reset  
    output reg  clk_div_tick // One-cycle pulse at divided rate  
);  
    int count;  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until \$finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module5/examples/<example>/ — run make run from that directory

Example directory layout

- dut/: spi_master.v, clk_div.v
- top_spi_baseline.sv: clk_div, spi_master; directed test in initial block
- sim_main.cpp: Clock, reset; run until \$finish

Makefile — spi_baseline run

```
# Makefile for Module 5 SPI baseline (basic testbench, no UVM)
#
# Usage:
#   make run
#

VERILATOR ?= verilator
TOPLEVEL ?= top_spi_baseline
DUT_SRC = dut/*.v
RTL_SRC = top_spi_baseline.sv $(DUT_SRC)
HARNESS = sim_main.cpp

VERILATOR_FLAGS = -sv --timing -Wall -Wno-fatal -Wno-TIMESCALEMOD -Wno-
WIDTHTRUNC

.PHONY: run compile clean
```

Key files to study

Open these in the repo

- ``module5/examples/spi_baseline/dut/spi_master.v`` — Mode 0 master
- ``module5/examples/spi_baseline/dut/clk_div.v`` — ``clk_div_tick``
- ``module5/examples/spi_baseline/top_spi_baseline.sv`` — directed test
- ``docs/SPI_LEARNING_GUIDE.md`` — protocol reference

Verification & testing methods

1. Baseline goals

- Confirm Mode 0 serialization before SPI UVM (Module 6).

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["start + data_in"] --> SPI["spi_master"]

SPI --> DONE["wait done"]

DONE --> PASS["baseline PASS"]

2. Directed test flow

- Pulse `start` + `data\_in` (0x55, 0xAA); `wait(done)`; read `[PASS]` messages.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["start + data_in"] --> SPI["spi_master"]

SPI --> DONE["wait done"]

DONE --> PASS["baseline PASS"]

3. Waveform learning (optional)

- Count 8 SCLK cycles per frame; verify MSB-first bit order on MOSI.
- Defer MISO, multi-slave CS, and random data to Module 6.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["start + data_in"] --> SPI["spi_master"]

SPI --> DONE["wait done"]

DONE --> PASS["baseline PASS"]

Directed SPI test

```
start    = 1'b0;
data_in  = 8'h00;

wait (rst_n === 1'b1);
repeat (10) @(posedge clk);

// Send 0x55
@(posedge clk);
start    = 1'b1;
data_in  = 8'h55;
@(posedge clk);
start    = 1'b0;
wait (done === 1'b1);
$display("[PASS] SPI baseline: 0x55 transfer done");

repeat (20) @(posedge clk);
```

...

Syllabus topics

1. SPI Protocol (Mode 0)

- Signals: sclk (serial clock), mosi (master out, slave in), cs_n (active-low chip select). Optional:...
- Mode 0 (CPOL=0, CPHA=0): SCLK idles low; data captured on rising edge; data changed on falling edge.
- Timing: One bit per clk_div_tick; clk_div_tick derived from system clock via divider (e.g. DIVIDER=...

2. Hands-on testbench (spi_baseline)

- Directed test in ``top_spi_baseline.sv``: ``start``,
``data_in``, ``wait(done)``.
- Pass: ``[PASS]`` lines and ``SPI baseline test PASS``.

Hands-on examples

Module 5 self-check

```
$ ./scripts/module5.sh --help
```

Terminal: module5_check

```
[[0;36m===== [[0m
[[0;36mModule 5: Demo commands (SPI baseline) [[0m
[[0;36m===== [[0m
```

From repo root:

```
# 1) Environment sanity:
verilator --version
make --version
```

```
# 2) Run the SPI baseline example (basic TB, no UVM):
cd module5/examples/spi_baseline
make run
```

```
[[1;33m[INFO] [[0m See module5/EXAMPLES.md and module5/examples/spi_baseline/README.md for more.
```

Example 1: SPI baseline master

- SPI Mode 0 master RTL (``spi_master`` + ``clk_div``)
- Directed writes 0x55 and 0xAA with ``start`` / ``wait(done)``
- Observing SCLK, MOSI, and CS_N timing in simulation (optional VCD)

Demo: SPI baseline master

```
$ cd module5/examples/spi_baseline && make run
```

Terminal: ex01_spi_baseline

Expected: `[PASS] SPI baseline: 0x55 transfer done`, `[PASS] SPI baseline: 0xAA transfer done`, then

Key concepts

1. Synchronous serial (SPI)

- ``sclk`` defines bit times; unlike UART there is an explicit clock line.
- Mode 0: idle low, sample MOSI on rising edge, change on falling edge.

2. Chip select frames

- ``cs_n`` low defines one transfer; typically 8 SCLK cycles per byte.

Common pitfalls

Wrong SPI mode in head

- Mistake: Sampling on falling edge when using Mode 0.
- Correct: Rising edge capture, falling edge change for CPOL=0, CPHA=0.
- Why: Bits will be shifted and MSB/LSB order will look wrong.

Trusting ``done`` only

- Mistake: No waveform or bus check on MOSI/SCLK.
- Correct: Optional VCD review or Module 6 monitor-based check.
- Why: ``done`` only means the FSM finished, not that bits were correct.

Practice & assessment

What you should know

- Describe SPI mode 0 and the role of sclk, mosi, cs_n, clk_div_tick.
- Explain the role of spi_master and clk_div.
- Run the spi_baseline example and interpret the directed test.
- Ready for Module 6: SPI UVM+SV verification.

Exercises

- Run spi_baseline
- Trace one transfer
- Mode 0 timing
- Optional: Waveforms

Assessment checklist

- Can describe SPI mode 0 (idle low, capture on rising, change on falling) and signals sclk, mosi, cs...
- Can explain the role of spi_master and clk_div in the RTL.
- Can run spi_baseline and interpret the directed test result.
- Ready for Module 6: SPI UVM+SV verification.

Summary & next steps

- Understand the SPI protocol (mode 0, signals), translate it to RTL (SPI master, clk_div), and verif...
- Complete module5/CHECKLIST.md
- Review module5/EXAMPLES.md and run each lab
- Next: SPI UVM+SV